

05506002 Methods of Numerical Computation

Lab 4

Objectives

1. Develop LU decomposition methods to decompose the coefficient matrix of the system of linear equations and calculate its inverse.
2. Develop Gauss-Seidel algorithm to iterative compute the solution of system of linear equations.

LU Decomposition

LU decomposition is a method to extract the coefficient matrix into L and U matrices to rearrange the left-hand side of the system to reduce the steps of calculation.

In Google Colab, create a function as follows.

```
import numpy as np

# LU Decomposition Without Pivoting
def lu_decomposition_no_pivot(A):
    n = A.shape[0]
    A = A.copy()

    steps = []

    for i in range(n - 1):
        for j in range(i + 1, n):
            factor = A[j, i] / A[i, i]

            A[j, i] = factor

            for k in range(i + 1, n):
                A[j, k] -= factor * A[i, k]

            steps.append((A.copy()))

    return A, steps
```

We can test the function by using example 4.1 of slide week 4.

```
A = np.array([
    [1, -2, 1],
    [2, 1, -3],
    [4, -7, 1]
], dtype=float)

b = np.array([0, 5, -1], dtype=float)

lu_no_pivot, steps_no_pivot = lu_decomposition_no_pivot(A)
print(np.array_str(lu_no_pivot, precision=3, suppress_small=True))

[[ 1.  -2.   1. ]
 [ 2.   5.  -5. ]
 [ 4.   0.2 -2. ]]
```

The steps of decomposition can be displayed as follows. Note that you can use the step displaying function we used last week.

```
display_steps(steps_no_pivot, title="Steps without Pivoting")
```

```
=====
Steps without Pivoting
=====
Step 1:
[[ 1. -2.  1.]
 [ 2.  5. -5.]
 [ 4. -7.  1.]]
-----
Step 2:
[[ 1. -2.  1.]
 [ 2.  5. -5.]
 [ 4.  1. -3.]]
-----
Step 3:
[[ 1.  -2.   1. ]
 [ 2.   5.  -5. ]
 [ 4.   0.2 -2. ]]
```

We can split this matrix into L and U matrices. We can test these matrices by multiplying them together to be back to the coefficient matrix.

```
def extract_lu(A_lu):
    n = A_lu.shape[0]
    L = np.eye(n) # Initialize L with identity matrix
    U = np.zeros_like(A_lu) # Initialize U as a zero matrix

    for i in range(n):
        for j in range(n):
            if i > j: # Below diagonal, belongs to L
                L[i, j] = A_lu[i, j]
            else: # Diagonal and above, belongs to U
                U[i, j] = A_lu[i, j]

    return L, U
```

```
L, U = extract_lu(lu_no_pivot)

print("\nLower Triangular Matrix L:")
print(np.array_str(L, precision=3, suppress_small=True))

print("\nUpper Triangular Matrix U:")
print(np.array_str(U, precision=3, suppress_small=True))

print("\nReconstructed Matrix L @ U:")
print(np.array_str(L @ U, precision=3, suppress_small=True))
```

```
Lower Triangular Matrix L:
[[1.  0.  0. ]
 [2.  1.  0. ]
 [4.  0.2 1. ]]
```

```
Upper Triangular Matrix U:
[[ 1. -2.  1.]
 [ 0.  5. -5.]
 [ 0.  0. -2.]]
```

```
Reconstructed Matrix L @ U:
[[ 1. -2.  1.]
 [ 2.  1. -3.]
 [ 4. -7.  1.]]
```

Using the LU matrix to solve the system of equations $[A]\{X\} = \{B\}$, we can divide it into two steps which are forward substitution to find D matrix in $[L]\{D\} = \{B\}$ and then use it in back substitution to find the solution in $[U]\{X\} = \{D\}$

```
def forward_substitution(L, b):

    n = len(b)
    D = np.zeros_like(b)

    for i in range(n):
        D[i] = b[i] - np.dot(L[i, :i], D[:i])

    return D

D = forward_substitution(lu_no_pivot, b)
print(np.array_str(D, precision=3, suppress_small=True))

[ 0.  5. -2.]
```

```
def backward_substitution(U, y):

    n = len(y)
    x = np.zeros_like(y)

    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - np.dot(U[i, i + 1:], x[i + 1:])) / U[i, i]

    return x

x = backward_substitution(lu_no_pivot, D)
print(np.array_str(x, precision=3, suppress_small=True))

[3. 2. 1.]
```

Note that the LU decomposition function above does not include pivot swapping. Therefore, there will be some systems that cannot be solved. We can reimplement the function as follows.

```
# LU Decomposition with Pivoting
def lu_decomposition_with_pivot(A):
    n = A.shape[0]
    P = np.eye(n) # Permutation matrix
    A = A.copy()

    steps = []
```

```

for i in range(n - 1):
    # Find the row with the largest pivot
    max_row = np.argmax(np.abs(A[i:, i])) + i
    if A[max_row, i] == 0:
        raise ValueError("No unique solution exists")
    # Swap rows
    A[[i, max_row]] = A[[max_row, i]]
    P[[i, max_row]] = P[[max_row, i]]

    for j in range(i + 1, n):
        factor = A[j, i] / A[i, i]

        A[j, i] = factor

        for k in range(i + 1, n):
            A[j, k] -= factor * A[i, k]

    steps.append((A.copy()))

return A, P, steps

```

This function will return the solution as well as the permutation matrix of how the rows were swapped. We can use the same system to test this function again.

```

A = np.array([
    [1, -2, 1],
    [2, 1, -3],
    [4, -7, 1]
], dtype=float)

b = np.array([0, 5, -1], dtype=float)

lu_with_pivot, P, steps_with_pivot = lu_decomposition_with_pivot(A)
print(np.array_str(lu_with_pivot, precision=3, suppress_small=True))
print(np.array_str(P, precision=3, suppress_small=True))

[[ 4.    -7.     1.    ]
 [ 0.5    4.5   -3.5   ]
 [ 0.25  -0.056  0.556]]
[[0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]]

```

```

L, U = extract_lu(lu_with_pivot)

print("\nLower Triangular Matrix L:")
print(np.array_str(L, precision=3, suppress_small=True))

print("\nUpper Triangular Matrix U:")
print(np.array_str(U, precision=3, suppress_small=True))

print("\nReconstructed Matrix L @ U:")
print(np.array_str(L @ U, precision=3, suppress_small=True))

print("\nPermuted Matrix P @ A:")
print(np.array_str(P @ A, precision=3, suppress_small=True))

Lower Triangular Matrix L:
[[ 1.  0.  0. ]
 [ 0.5 1.  0. ]
 [ 0.25 -0.056 1. ]]

Upper Triangular Matrix U:
[[ 4.  -7.  1. ]
 [ 0.  4.5 -3.5 ]
 [ 0.  0.  0.556]]

Reconstructed Matrix L @ U:
[[ 4. -7.  1.]
 [ 2.  1. -3.]
 [ 1. -2.  1.]]

Permuted Matrix P @ A:
[[ 4. -7.  1.]
 [ 2.  1. -3.]
 [ 1. -2.  1.]]

```

When we decompose the coefficient matrix into LU matrix, we can use this matrix to calculate the inverse of the matrix as follows.

```

def inverse(A, P):
    n = A.shape[0]

    # Initialize the inverse matrix
    A_inv = np.zeros_like(A)

    # Compute each column of the inverse
    for j in range(n):
        # Standard basis vector e_j
        e_j = np.zeros(n)
        e_j[j] = 1

        # Forward substitution: L * y = P * e_j
        y = forward_substitution(A, P @ e_j)

        # Backward substitution: U * x = y
        x = backward_substitution(A, y)

        # Store the result as the j-th column of the inverse
        A_inv[:, j] = x

    return A_inv

```

We will use the LU matrix before to calculate the inverse of the matrix.

```

A_inv = inverse(lu_with_pivot, P)

print("\nInverse of A (A^-1):")
print(np.array_str(A_inv, precision=3, suppress_small=True))

# Verify the result: A @ A^-1 should be the identity matrix
I = A @ A_inv
print("\nVerification (A @ A^-1):")
print(np.array_str(I, precision=3, suppress_small=True))

Inverse of A (A^-1):
[[ 2.   0.5 -0.5]
 [ 1.4  0.3 -0.5]
 [ 1.8  0.1 -0.5]]

Verification (A @ A^-1):
[[ 1.  0.  0.]
 [-0.  1.  0.]
 [-0.  0.  1.]]

```

Gauss-Seidel

Gauss-Seidel algorithm is an iterative algorithm to find the solution of a system of equations. We can create a function as follows.

```
def gauss_seidel(A, b, x0=None, es=0.01, max_iter=10000):
    n = A.shape[0]
    x = np.zeros_like(b) if x0 is None else x0.copy()
    iter_count = 0

    errors = [] # List to store approximation errors
    ea = 100 # Initial approximation error

    while iter_count < max_iter:
        old_x = x.copy()
        iter_count += 1

        for i in range(n):
            # Compute the sum for the Gauss-Seidel formula
            sum_1 = np.dot(A[i, :i], x[:i]) # Terms involving updated x
            sum_2 = np.dot(A[i, i+1:], old_x[i+1:]) # Terms involving old
x
            x[i] = (b[i] - sum_1 - sum_2) / A[i, i]

        # Check for convergence
        ea = np.linalg.norm(np.abs(x - old_x)/x * 100, ord=np.inf)
        errors.append(ea)

        if (ea.max() < es):
            return x, iter_count, errors

    raise ValueError("Gauss-Seidel method did not converge within the maximum number of iterations.")
```

This function defines the max number of iterations at 10000 and the specific error at 0.01. You can change these numbers as desired.

We can test the algorithm using the following system of equations.


```

A = np.array([
    [4, -1, 2],
    [1, 5, 1],
    [2, 1, 3]
], dtype=float)

b = np.array([8, 7, 9], dtype=float)

# Solve using Gauss-Seidel
# Initial guess
x0 = np.array([0, 0, 0], dtype=float)

try:
    solution, iterations, errors = gauss_seidel(A, b, x0=x0)
    print(f"Root: {solution}")
    print(f"Iterations: {iterations}")
    print(f"Approximation Errors: {errors}")
except ValueError as e:
    print("Error without pivoting:", e)

Root: [1.23080539 0.76923655 1.92305089]
Iterations: 12
Approximation Errors: [100.0, 26.315789473684198, 15.736040609137063,
6.381063810638109, 2.6469315702519993, 1.0678088954861955,
0.42914647220765945, 0.17193788044821154, 0.06882380798180358,
0.02753699378847486, 0.011016020092664782, 0.0044066014444473205]

```

Tasks (Submit through Google Classroom)

Use LU decomposition to the solution of the following situation.

You and your friends go out for a meal. Each person orders a combination of three food items:

- Main Dish
- Drinks
- Desserts

At the end of the meal, each person contributes a total amount to pay for their share of the food as follows.

- Person 1 ordered 2 main dishes, 3 drinks, and 1 dessert. He paid 230 baht

- Person 2 ordered 1 main dish, 2 drinks, and 2 desserts. He paid 150 baht
- Person 3 ordered 1 main dish, 1 drink, and 3 desserts. He paid 130 baht

Your task is to determine the **cost per item** (i.e., the price of a Main Dish, Drink, and Dessert) based on what each person ordered. Show the entire steps in Python.